PX-1A — Dataset Usage Fix

Philosophy: Datasets assist, they don't dominate.

Root Cause

`pickDataset()` had two issues:
1. **Always fell back to** `datasets[0]` even when nothing matched — so every scenario got a dataset whether it fit or not.
2. **Every scenario inherited the requirement title** → SQL/XSS/boundary scenarios matched "employee" → all pointed at `new_employee`.

Net effect: security and boundary scenarios (which should use crafted payloads) were being handed a valid employee record, and datasets dominated the output instead of assisting it.

The Fix

1. Priority order — trust the DATA before the LABEL

The scoring order now reflects how reliable each signal actually is:

```
Dataset KEYS         (weight 3)   ← primary   — the real data structure
Dataset NAME         (weight 1)   ← secondary — a human-chosen label
Requirement context  (tie-break)  ← tertiary  — never selects on its own
(no fit) → undefined → inline values
```

A dataset's **keys** are the strongest signal because names are inconsistent (a `Regression_Set` can still hold employee keys), whereas keys describe what the data actually is. This corrects the earlier draft, which weighted the **name** 2× higher than keys — the opposite of the reliable order.

```
const KEY_WEIGHT = 3;  // a dataset KEY match — the most reliable signal
const NAME_WEIGHT = 1; // a dataset NAME match — a weaker, human-chosen label
```

2. Requirement title is a TIE-BREAKER only

The requirement title used to boost every positive scenario. It is now scored into a separate `tiebreak` bucket that never contributes to the primary score, so it can only choose between datasets that already fit structurally — it can never pull in a dataset on its own.

```
const primary  = scorePool(keyTokens, nameTokens, [...fieldTerms, ...scenarioTerms]);
const tiebreak = scorePool(keyTokens, nameTokens, requirementTerms); // never in
`primary`
// sort: primary desc → tiebreak desc → upload order asc  (deterministic)
```

3. Whole-token matching, not substrings

Substring matching leaked common words: the scenario word **“data”** substring-matched a dataset named **“Dataset_A”**. Matching is now on whole tokens (lowercased, split on non-alphanumeric, length ≥ 3).

```
const nameTokens = new Set(toTerms(d.name ?? ''));
const keyTokens  = new Set((d.sampleKeys || []).flatMap(k => toTerms(k)));
if (keyTokens.has(t)) s += KEY_WEIGHT; else if (nameTokens.has(t)) s += NAME_WEIGHT;
```

4. Only data-consuming scenarios attempt a match

Because security/boundary scenarios share the same form, their field terms would otherwise match the dataset keys. `scenarioConsumesDataset()` gates this: positive creates and the duplicate check consume a dataset; SQL, XSS, whitespace, boundary, numeric, unicode, special-char and length scenarios never even try — they fall straight to inline values.

5. Transparent, simpler wording

Users see exactly where values come from — and the “no match” case is worded plainly:

```
const testData = dataset?.name
  ? `✓ Dataset: ${dataset.name} (keys: ...)`
  : 'Generated sample values';   // was: "Generated inline values (no matching data-
set)"
```

Proof (Add New Employee requirement)

`new_employee` and `duplicate_employee` share identical keys `[firstName, lastName]`, so they tie on structural fit — the differentiator is the NAME token (“duplicate”) and the requirement tie-break, exactly as intended.

Scenario	Test Data Field	Correct?
Positive “Create a record with valid data”	`✓ Dataset: new_employee (keys: firstName, lastName)`	✅
Duplicate “Re-submitting does not create duplicate”	`✓ Dataset: duplicate_employee (keys: firstName, lastName)`	✅
SQL “SQL-injection input is rejected”	`Generated sample values`	✅
XSS “XSS payload is escaped”	`Generated sample values`	✅
Whitespace “Whitespace-only fields rejected”	`Generated sample values`	✅
Boundary “Field length boundaries”	`Generated sample values`	✅

Priority proofs (`scripts/intent-proof.ts`)

1. **Keys beat name** — a dataset whose keys fit wins over one whose name merely mentions the requirement. ✅
2. **Name is only secondary** — with equal key fit, the closer name wins. ✅
3. **Meaningless names fall back** — `Dataset_A` (no key/name fit) → inline values, not a spurious “data” substring match. ✅
4. **Requirement context only breaks ties** — it never selects a structurally-unfit dataset on its own. ✅

Product Rules Implemented

✅ **Datasets assist, they don’t dominate** — no fit → inline values
✅ **Keys are the primary signal** (weight 3), name secondary (weight 1)
✅ **Requirement title is a tie-breaker only** — never selects alone
✅ **Dataset usage is visible** (transparency builds trust)
✅ **Deterministic tie-breaking** (earliest upload wins)
✅ **Works with meaningless dataset names** (Dataset_A → inline values)

Matching Strategy (documented in code)

LEXICAL, not semantic — and intentionally so.

INTENTIONALLY OUT OF SCOPE (future improvement, not this PR):
- Semantic matching (e.g. “Employee_Master” ≈ “Add Employee” without lexical overlap)

- Domain-aware matching (e.g. HR domain → prefer HR-related datasets)
- Multi-word compound matching (e.g. “new employee” as a phrase)

Better to generate deterministic inline data than to randomly guess a dataset.

Scope

Strictly **dataset selection only**. Expected Results, Steps, and the Planner are untouched by this change.

Files Changed

`src/engines/scenario-builder.ts`:
- `pickDataset()` — rewritten to take `DatasetMatchSignals { fieldTerms, scenarioTerms, requirementTerms }`; keys weighted 3, name 1, requirement as tie-break; whole-token matching; returns `undefined` when nothing fits.
- `scenarioConsumesDataset()` — new gate: only positive creates and duplicate checks attempt a dataset match.
- Test Data field — transparent wording; “Generated sample values” when no dataset fits.

Tests

- `tests/unit/dataset-usage-priority.test.ts` — **14/14 pass** ✅ (durable, locks keys-primary priority + token matching)
- `scripts/intent-proof.ts` — **4/4 priority proofs pass** ✅
- `scripts/intent-gate0.ts` — Gate 0 reproduction now resolves correctly ✅
- `scenario-builder` + related suites — **44/44** and **83/83 pass** ✅
- `tsc --noEmit` — clean ✅